

DA5300: Introduction to Python Programming

Laboratory

Laboratory 1 – Object-Oriented Programming using a Cricket Database

Objective

In this laboratory you will design a small cricket database management system using Python. The objective is to gradually introduce the following Python programming concepts.

- Strings
- Lists
- Tuples
- Dictionaries
- File I/O
- Classes and Objects
- Inheritance
- Composition
- Class Methods
- Iterators
- Generators

Background

A text file named `cricket.txt` is provided.

The file contains information about international cricket players. Each record corresponds to exactly one player playing one format of the game.

The file is intentionally unordered.

Records belonging to the same player or the same country are not grouped together.

A player may appear

- once (plays one format),
- twice (plays two formats), or
- three times (plays all three formats).

The three formats are

- TEST

- ODI
- T20

Each record contains

- Player Name
- Country
- Format
- Batting Average
- Batting Strike Rate
- Bowling Average
- Bowling Economy

Your task is to organize the information into suitable Python classes.

Sample Database

The following shows a small portion of the supplied file.

```
PLAYER=Virat Kohli
TEAM=India
FORMAT=ODI
BAT_AVG=58.1
BAT_SR=93.6
BOWL_AVG=999
BOWL_ECO=999
```

```
PLAYER=Glenn Maxwell
TEAM=Australia
FORMAT=T20
BAT_AVG=30.4
BAT_SR=154.6
BOWL_AVG=22.8
BOWL_ECO=7.2
```

```
PLAYER=Joe Root
TEAM=England
FORMAT=TEST
BAT_AVG=50.2
BAT_SR=57.8
BOWL_AVG=45.0
BOWL_ECO=3.1
```

```
PLAYER=Virat Kohli
TEAM=India
FORMAT=T20
BAT_AVG=52.8
BAT_SR=138.1
```

BOWLAVG=999
BOWLECO=999

PLAYER=Rashid Khan
TEAM=Afghanistan
FORMAT=T20
BAT_AVG=18.5
BAT_SR=153.7
BOWLAVG=15.6
BOWLECO=6.2

PLAYER=Jasprit Bumrah
TEAM=India
FORMAT=TEST
BAT_AVG=8.2
BAT_SR=47.1
BOWLAVG=20.3
BOWLECO=2.65

PLAYER=Steve Smith
TEAM=Australia
FORMAT=TEST
BAT_AVG=58.6
BAT_SR=54.2
BOWLAVG=999
BOWLECO=999

Cricket Database

Each country maintains three independent squads.

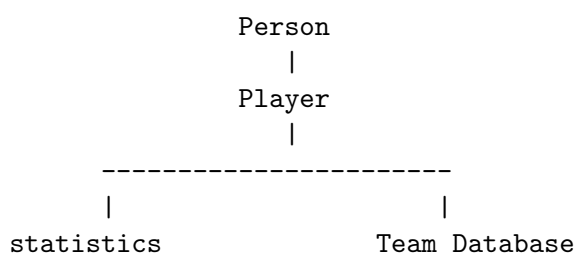
- Test Squad
- ODI Squad
- T20 Squad

Each squad may contain more than eleven players.

When a playing XI is requested, the program should randomly generate eleven distinct players from the corresponding squad.

Class Design

Design the following classes.



Statistics

Team

The relationships are

- Player inherits from Person.
- Player contains one Statistics object for every format played.
- Team contains three squads.

Part I – File Processing

Q1.1 Read the complete file into memory.

Q1.2 Count the total number of records.

Q1.3 Print all unique player names.

Q1.4 Print all unique countries.

Q1.5 Count the number of TEST, ODI and T20 records.

Q1.6 Determine the number of players who play

- only TEST
- only ODI
- only T20
- exactly two formats
- all three formats

Part II – Lists, Tuples and Dictionaries

Q2.1 Store all countries in a list.

Q2.2 Store all player names in another list.

Q2.3 Construct

```
country_players[country]
```

which stores the list of players belonging to that country.

Q2.4 Construct

```
player_formats[player]
```

whose value is a tuple containing the formats played by the player.

Q2.5 Construct

```
country_squads[country] ["TEST"]
```

```
country_squads[country] ["ODI"]
```

```
country_squads[country] ["T20"]
```

where every entry stores a list of players.

Q2.6 Verify that every player appears in the correct squad(s).

Part III – Statistics Class

Design a class named

Statistics

The class stores

- Batting Average
- Batting Strike Rate
- Bowling Average
- Bowling Economy

Implement the following member functions.

Q3.1 Constructor

Q3.2 Display statistics

Q3.3 Modify batting average

Q3.4 Modify batting strike rate

Q3.5 Modify bowling average

Q3.6 Modify bowling economy

Q3.7 Reset all statistics

Part IV – Person Class

Implement the class

Person

containing

- Name
- Country

Implement

Q4.1 Constructor

Q4.2 Display function

Q4.3 Update country

Part V – Player Class

The Player class must inherit from Person.

Each Player object stores

- Name
- Country
- Statistics corresponding to TEST
- Statistics corresponding to ODI
- Statistics corresponding to T20

Store the statistics as

```
statistics = {  
  
    "TEST" : Statistics(...),  
  
    "ODI" : Statistics(...),  
  
    "T20" : Statistics(...)  
  
}
```

Implement the following member functions.

- Q5.1** Add statistics for a format.
- Q5.2** Delete statistics of a format.
- Q5.3** Modify statistics of a format.
- Q5.4** Check whether the player plays a given format.
- Q5.5** Return the list of formats played.
- Q5.6** Display complete player information.

““latex

Part VI – Team Class

Implement the class

Team

Each Team object stores

- Team Name
- Test Squad
- ODI Squad

- T20 Squad

Each squad stores a list of **Player** objects.
Implement the following member functions.

Q6.1 Constructor.

Q6.2 Add a player to a squad.

Q6.3 Delete a player from a squad.

Q6.4 Transfer a player from one squad to another.

Q6.5 Search for a player by name.

Q6.6 Return the Test squad.

Q6.7 Return the ODI squad.

Q6.8 Return the T20 squad.

Q6.9 Return the number of players in a squad.

Q6.10 Print all players available in all three formats.

Q6.11 Print all players available in exactly one format.

Q6.12 Print players common to two squads.

Q6.13 Print players belonging to one squad but not another.

Q6.14 Return the total number of unique players in the team.

Part VII – Random Playing XI

Each squad may contain more than eleven players.
Implement the following member function.

`generate_playing_xi(format)`

The function should

- randomly select eleven distinct players,
- ensure no player is repeated,
- print the selected Playing XI.

Execute the function multiple times and verify that different valid Playing XIs are generated.

Part VIII – Ordinary Functions versus Class Methods

Implement the following as ordinary Python functions.

Q8.1 Print all teams.

Q8.2 Print all players.

Q8.3 Print all players belonging to a team.

Q8.4 Print the Test squad.

Q8.5 Print the ODI squad.

Q8.6 Print the T20 squad.

Now implement the same functionality using class methods.

Comment on the advantages and disadvantages of the two approaches.

Part IX – File I/O

Implement the following.

Q9.1 Read the supplied file `cricket.txt`.

Q9.2 Construct all `Player`, `Statistics` and `Team` objects.

Q9.3 Save the organized database into

`organized_cricket.txt`

Q9.4 Read the organized database back into memory and verify that it contains the same information.

Part X – Iterator

Modify the `Team` class so that it becomes iterable.

The following code should work.

```
india = Team(...)

for player in india:

    print(player.name)
```

The iterator should iterate over every unique player in the team, irrespective of the format(s) played.

Part XI – Generators

Implement generators for the following.

- Q11.1** Yield all players in the Test squad.
- Q11.2** Yield all players in the ODI squad.
- Q11.3** Yield all players in the T20 squad.
- Q11.4** Yield players who play only Test cricket.
- Q11.5** Yield players who play only ODI cricket.
- Q11.6** Yield players who play only T20 cricket.
- Q11.7** Yield players who play all three formats.
- Q11.8** Yield players having batting average greater than a user supplied value.
- Q11.9** Yield players having bowling average less than a user supplied value.
- Q11.10** Yield all players of a given country one at a time.

Part XII – Updating the Database

Implement a menu-driven interface supporting the following operations.

- Q12.1** Add a new team.
- Q12.2** Delete a team.
- Q12.3** Add a player.
- Q12.4** Delete a player.
- Q12.5** Add a player to the Test squad.
- Q12.6** Add a player to the ODI squad.
- Q12.7** Add a player to the T20 squad.
- Q12.8** Remove a player from a squad.
- Q12.9** Transfer a player from one squad to another.
- Q12.10** Add statistics for a new format.
- Q12.11** Delete statistics corresponding to a format.
- Q12.12** Modify batting average.
- Q12.13** Modify batting strike rate.
- Q12.14** Modify bowling average.
- Q12.15** Modify bowling economy.
- Q12.16** Save the updated database.

Part XIII – Testing

Demonstrate the correctness of your implementation using suitable test cases.
Your demonstration should include

- Q13.1** Reading the supplied database.
- Q13.2** Creating Team, Player and Statistics objects.
- Q13.3** Adding a new player.
- Q13.4** Deleting an existing player.
- Q13.5** Updating player statistics.
- Q13.6** Generating Playing XI for each format.
- Q13.7** Saving the modified database.
- Q13.8** Reading the saved database again.

Bonus Problems

Attempt any five of the following.

- B1.** Print the top five batters in each format.
- B2.** Print the five best bowlers in each format.
- B3.** Find the player who plays the maximum number of formats.
- B4.** Find the country with the largest Test squad.
- B5.** Find the country with the largest ODI squad.
- B6.** Find the country with the largest T20 squad.
- B7.** Export the database as a CSV file.
- B8.** Export the database as a JSON file.
- B9.** Sort players using batting average.
- B10.** Sort bowlers using bowling average.
- B11.** Overload the comparison operators for Player objects.
- B12.** Implement `__str__()` for all classes.
- B13.** Implement `__repr__()` for all classes.
- B14.** Implement `__len__()` for the Team class so that

`len(india)`

returns the number of unique players.
- B15.** Implement `__contains__()` so that

```
"Virat Kohli" in india
```

returns True if the player belongs to any squad.

B16. Implement `__getitem__()` so that

```
india["Virat Kohli"]
```

returns the corresponding Player object.

B17. Implement `__iter__()` without storing duplicate Player objects.

B18. Implement a generator that yields all left-handed batters (assume the handedness information is available).

B19. Display the Playing XI in batting order.

B20. Compute the average batting average and bowling average for each team and each format.

the features implemented.